

Autonomous Obstacle Avoiding Rover

Abdurakhman Aidarkhan, Aleksei Lukianov, Azamat Zhalgassov, Said Darkhanuly

Hof University of Applied Sciences, Hof 95028 Germany

Abstract: The topic of our project is development of an autonomous rover which is able to detect obstacles using LIDAR environment scans and avoid them in case some specific safe distance is reached. Entire decision-making logic is written on Python programming language with support of several required libraries. Additionally, the rover is capable of solving mazes using the "right-hand rule". Our project goal: the rover successfully avoids obstacles and solves mazes autonomously.

1. Waveshare UGV02

To begin with, as a basis of the project Waveshare UGV02 mobile robot was used. Along with other components, this hardware is dedicated to accomplish maze solving and obstacle avoiding in the context of our project.

UGV02 is a 4WD six-wheel mobile robot chassis with superb off-road performance, shock absorbers and open-source code that supports customization development. It supports external host computers (Raspberry Pi, Jetson Nano, RDK X3, etc.), and the host computer communicates with the ESP32 slave computer through the serial port [1]. Also, three 18650 batteries were used as an external source of power for the UGV02.

1.1. UGV02 features

- 1) The use of flexible rubber tires substantially reduces the impact of complex terrain on the product.
- 2) With a 4WD six-wheel drive system, it has excellent climbing ability.
- 3) Equipped with an interactive device 0.96-inch OLED screen.
- 4) ESP32 automatically sets a WI-FI hotspot when it is turned on. You can use a computer (Linux/Windows/Mac) to access the control web page using a browser.
- 5) Can be extended with a variety of host computers, use the HTTP requests to transmit data in JSON format to control.
- 6) Most of the configuration settings can be realized by JSON commands (for example, it is possible to configure WI-FI connection for the product without connecting the USB cable)

2. ESP32 microcontroller

In the beginning, we started with factory setting restoring and firmware uploading to the Waveshare UGV02, which is based on the ESP32 microcontroller. ESP32 is a microcontroller controlling the operation of all the main components of the robot and the overall behaviour of the system.

The ESP32 is a 32-bit microcontroller with integrated Wi-Fi, Bluetooth, and BLE functionality, designed for a wide range of applications. It is a series of energy-efficient and low-cost devices developed by Espressif Systems [2].

2.1. Features of ESP-Wroom-32

- 1) The ESP-WROOM-32 contains a low-power Tensilica Xtensa® Dual-Core 32-bit LX6 micro-processor operating at 240 MHz (994.26 CoreMark; 4.14 CoreMark/MHz).
- 2) 448 KB of ROM for booting and core functions.
- 3) 520 KB of on-chip SRAM for data and instructions.
- 4) 4 MB of flash memory.
- 5) 16 KB of SRAM in the RTC.
- 6) Wi-Fi 802.11 b/g/n.
- 7) Bluetooth v4.2 (BR/EDR and Bluetooth Low Energy).

3. Process of factory setting restoring

To accomplish this, the mobile robot was partially disassembled, after which the USB port on the control board was connected to the computer via USB cable. Following that, the **ESP32 download tool for UGV02** was downloaded and used, which is designed for factory setting restoring and firmware updating. After launching the utility, "Chip Type" was selected as ESP32 and "WorkMode" as Factory. With these settings, the relative path will be used when calling the binary file, so it is not necessary to manually enter the binary file path.

During this process, the COM port was selected and the data transfer rate (BAUD) was set to 921600 for accelerating the firmware loading process. Once it was completed, UGV02 was ready for further use.

4. Nvidia Jetson Xavier NX

Since UGV02 is supposed to avoid obstacles and solve maze autonomously and without external control, we decided to utilize the Nvidia Jetson Xavier NX as a basis of decision-making logic implementation.

The NVIDIA Jetson Xavier NX is a high-performance embedded computing platform designed for edge computing and autonomous systems. It is based on a heterogeneous system-on-chip (SoC) architecture that integrates multiple specialized processors optimized for various workloads. The module is equipped with a 6-core 64-bit ARMv8.2 processor, which delivers efficient general-purpose processing with low power consumption, making it suitable for mobile and embedded applications [3], [4].

4.1. Jetson Xavier flash

Jetson Xavier NX runs a Linux-based operating system and supports the NVIDIA JetPack software development kit, which includes CUDA, cuDNN, TensorRT and other acceleration libraries. Prior to utilizing this hardware component, it was accomplished the reflash process. For this purpose NVIDIA SDK Manager was installed on a host PC running Ubuntu 20.04, as this version of the operating system is required for compatibility with the Jetson flashing tools.

NVIDIA SDK Manager is the official tool provided by NVIDIA for installing software on Jetson devices. Following that, Jetson Xavier NX was connected to the host PC via USB A to Micro USB cable and switched to recovery mode using jumper. During the reflashing process, the JetPack 5.0.2 software package was selected, which is a software development kit that consists of the Linux operating system, hardware drivers and optimized libraries for GPU acceleration. After the process was completed, the Nvidia Jetson Xavier NX was ready for integration into the project robotic system.

5. Python as the main programming language

Delving into the project robotic system, previously reflashed Jetson Xavier is responsible for decision-making logic execution, which is written on Python with the support of required libraries (rplidar, requests, time). Each of these libraries are essential for performing specific tasks in the context of our robotic system.

Python was selected due to its readability, strong support for robotics and extensive ecosystem. To support the execution of the core logic, a Python environment was installed and configured on the Jetson device. In addition, the required rplidar library was installed to enable environmental data processing, which is needed for determining further actions based on situation (for example, presence of obstacles).

6. RPLIDAR A1

RPLIDAR A1 can perform a 360-degree scan within a 12-meter range. The produced 2D point cloud data can be used in mapping, localization and environment modeling. The scanning frequency of RPLIDAR A1 reached 5.5hz when sampling 1450 points each round. And it can be configured up to 10hz.

RPLIDAR A1 is a laser triangulation measurement system. It can work excellently in all kinds of indoor environments and outdoor environments without direct sunlight exposure. Additionally, it contains a range scanner system and a motor system. After powering on each sub-system, it starts rotating and scanning clockwise. Users can get range scan data through the communication interface (Serial port/USB) [5].

6.1. Features of RPLIDAR A1

- Laser triangulation principle.
- Support UART communication.
- The measuring radius is 12m.
- 8000Hz measurement frequency.
- 360° scanning range.
- Laser Safety Meets Class 1 Standards.

7. External Python libraries used

7.1. rplidar library

In order to work with this component, the RPLIDAR library is needed for communication with RPLIDAR A1 laser scanning sensors. It enables device initialization, motor control and constant receiving of distance and angle measurements obtained from 360-degree laser scans. The library abstracts low-level data and delivers it in structured form that can be directly used for obstacle detection and environment modeling.

After importing the required library and initializing the device, a normalization function was implemented to map all angles into the range from -180° to 180° . While the program is running, the system continuously receives scan data consisting of a set of distance measurements associated with specific angles. Based on the normalized angle values, the scan data is divided into pre-defined spatial regions. The frontal region is defined within the angular range from -15° to 15° , while the right-side region covers angles from 65° to 115° . For each region, the minimum distance value is extracted in order to determine the closest obstacle in that direction. The algorithm then compares the obtained distances with a predefined safe distance using conditional statements (`if`, `elif`, `else`). Based on this comparison, the appropriate control decision is determined and transmitted to the UGV02 in JSON format.

Since the Jetson Xavier NX communicates with the UGV02 by sending control commands (in JSON format) over the HTTP protocol, we used functions for stopping both the scanning process and the motor rotation of the RPLIDAR A1. These steps were accomplished to ensure that only a single scan was used for each decision, preventing constant scanning and possible buffer overflow of the scanning device while a control decision is computed.

7.2. time library

Also, we utilized the **sleep** function from the **time** library after transmission of each control command over HTTP protocol. The sleep function is provided by Python's built-in time module. Before using it, you must first import the module into your script. Once imported, the function can be called using the `time.sleep()` syntax. The function accepts a single argument that specifies how long execution should pause. This value is expressed in seconds and can be either an integer or a floating-point number.

When the statement `time.sleep(t)` is executed, the program pauses for `t` seconds before moving on to the next line of code [6]. In our case, It was necessary to guarantee that each HTTP request was processed by the ESP32 controller completely before sending the next request.

7.3. requests library

As mentioned before, for sending HTTP requests a **requests** library was used. The requests library is a Python module for sending HTTP requests simply. It provides functions to perform GET, POST and other HTTP methods, allowing programs to communicate with web servers or networked devices [7]. The library abstracts low-level protocol details such as connection handling, headers and data encoding. In the robotics context, requests can be used to transmit control commands and receive responses.

8. Small network creation and ssh protocol using

Although it is possible to connect UGV02 with Jetson Xavier NX via **40 pin cable** for communication purposes, our team decided to use HTTP requests. First of all, a small network was created and the UGV02 was connected to an external access point using a specific JSON command. Following that, the external host became able to be in the same network with the UGV02 and execute Python code on the Jetson Xavier remotely, as well as manage its overall behaviour. For this purpose our team used **ssh protocol** and configured a **static ip address** on the Jetson host for convenient and safe **ssh** connection in the future. Since we were forced to use a small network, HTTP requests were used to avoid using extra hardware for the physical connection between mobile robot and Jetson host.

9. Jetson external power source

9.1. AGM accumulator

To be completely autonomous, there should be an external power for jetson computer. UGV's accumulators can't be used for several reasons: low capacity of batteries results in very short working time, initial amperage of the whole system can be high and cause jetson reboot.

Our team decided to split power sources of the rover and the jetson. AGM accumulator was chosen.

AGM battery is a type of lead-acid battery. AGM batteries differ from flooded lead-acid batteries in that the electrolyte is held in the glass mats, as opposed to freely flooding the plates. Very thin glass fibers are woven into a mat to increase the surface area enough to hold a sufficient amount of electrolyte on the cells for their lifetime. The fibers that compose the fine glass mat do

not absorb and are not affected by the acidic electrolyte. These mats are wrung out 2–5% after being soaked in acids just prior to finish manufacturing [8], [9].

In our case Jetson has not so much load, so can easily work in 15 W mode. Jetson supports power input between 9 and 19 volts. Our battery has fixed 12 V.

It is necessary to know amperage of the system to understand lifetime of the battery and decide which fuse should be implemented in the system for protection against short circuits.

Amperage can be calculated using this formula:

$$I = \frac{W}{U} \quad (1)$$

Substituting the known values:

$$I = \frac{15 \text{ W}}{12 \text{ V}} = 1.25 \text{ A} \quad (2)$$

According to this data, life of the battery can be calculated as:

$$T = \frac{C \times U}{I \times U} = \frac{C}{I} \quad (3)$$

$$T = \frac{7.2 \text{ Ah}}{1.25 \text{ A}} = 5.76 \text{ h} \quad (4)$$

In practice, Jetson uses only 6W, so we can adjust calculations.

$$I = \frac{6 \text{ W}}{12 \text{ V}} = 0.5 \text{ A} \quad (5)$$

$$T = \frac{7.2 \text{ Ah}}{0.5 \text{ A}} = 14.4 \text{ h} \quad (6)$$

Efficiency of any hardware can't be 100%. According to the tests, battery lasts for about 10 hours, which is 70% efficiency rate.

9.2. Protective fuse

To prevent the whole system from shortcut circuits protective fuse should be implemented.

A fuse is an electrical safety device that operates to provide overcurrent protection of an electrical circuit. Its essential component is a metal wire or strip that melts when too much current flows through it, thereby stopping or interrupting the current. It is a sacrificial device; once a fuse has operated, it is an open circuit, and must be replaced or rewired, depending on its type. Today there are thousands of different fuse designs which have specific current and voltage ratings, breaking capacity, and response times, depending on the application. The time and current operating characteristics of fuses are chosen to provide adequate protection without needless interruption. Wiring regulations usually define a maximum fuse current rating for particular circuits. A fuse can be used to mitigate short circuits, overloading, mismatched loads, or device failure. When a damaged live wire makes contact with a metal case that is connected to ground, a short circuit will form and the fuse will melt.

A fuse consists of a metal strip or wire fuse element, of small cross-section compared to the circuit conductors, mounted between a pair of electrical terminals, and usually enclosed by a non-combustible housing. The fuse is arranged in series to carry all the charge passing through

the protected circuit. The resistance of the element generates heat due to the current flow. The size and construction of the element is empirically determined so that the heat produced for a normal current does not cause the element to attain a high temperature. If too high of a current flow, the element rises to a higher temperature and either directly melts, or else melts a soldered joint within the fuse, opening the circuit.

The fuse element is made of zinc, copper, silver, aluminum, or alloys among these or other various metals to provide stable and predictable characteristics. The fuse ideally would carry its rated current indefinitely, and melt quickly on a small excess. The element must not be damaged by minor harmless surges of current, and must not oxidize or change its behavior after possibly years of service.

Our system consumes not more than 1.5 A, accordingly 3 A fuse will be enough.

10. Core algorithms

10.1. Maze-solving algorithm

The rover is expected to solve simply-connected maze, accordingly any algorithm can be implemented. In our case we have chosen right-hand rule [10].

If the maze is simply connected, that is, all its walls are connected together or to the maze's outer boundary, then by keeping one hand in contact with one wall of the maze the solver is guaranteed not to get lost and will reach a different exit if there is one; otherwise, the algorithm will return to the entrance having traversed every corridor next to that connected section of walls at least once. The algorithm is a depth-first in-order tree traversal. Another perspective into why wall following works is topological. If the walls are connected, then they may be deformed into a loop or circle. Then wall following reduces to walking around a circle from start to finish. To further this idea, notice that by grouping together connected components of the maze walls, the boundaries between these are precisely the solutions, even if there is more than one solution.

If the robot sees free space on the right, he turns right and moves for 1 cell; if there is an obstacle on the right and no obstacles in front of it, it moves forward for 1 cell; if there are obstacles both on the right and in front, the rover turns left.

10.2. Obstacle avoidance algorithm

Another mode of the rover is obstacle avoidance. Algorithm is rather simple. The rover moves forward until it sees an obstacle in front of it, then we transmit emergency stop command. The robot scans the environment and decides on which side is more free space to go; then it turns and moves for 1 cell; after that the rover turns in the opposite direction, so it continues moving in the original direction until it sees new obstacle.

11. Work division

- Hardware setup made by Abdurakhman Aidarkhan
- Jetson external powerset made by Aleksei Lukianov
- Python files written by Abdurakhman Aidarkhan and Aleksei Lukianov
- Cardboard maze for tests made by Azamat Zhalgassov
- Network setup made by Said Darkhanuly
- Presentation made by Azamat Zhalgassov and Said Darkhanuly

12. Conclusion

Our initial goal was successfully reached and the mobile robot is capable of maze solving and obstacle avoiding. However, we faced some challenges, such as not ideal surface.

As we mentioned in our pager, path memorization and optimization could be added for more efficient maze passing without environment scanning. In addition, utilized Nvidia Jetson Xavier NX is designed to handle high AI workloads due to embedded accelerators. Therefore, AI can be integrated into our project in the future. published results.

References

- [1] "Waveshare ugv02," <https://www.waveshare.com/wiki/UGV02>.
- [2] "Introduction to esp32," <https://www.electronicwings.com/esp32/introduction-to-esp32>.
- [3] "Jetson xavier nx," <https://developer.nvidia.com/blog/jetson-xavier-nx-the-worlds-smallest-ai-supercomputer/>.
- [4] "Nvidia jetson xavier," <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-xavier-series/>.
- [5] "Rplidar a1," https://www.waveshare.com/wiki/RPLIDAR_A1.
- [6] "Python time.sleep(): Pausing code execution in python," <https://mimo.org/glossary/python/time-sleep>.
- [7] "Python's requests library (guide)," <https://realpython.com/python-requests/>.
- [8] "Battery charging - agm," <https://support.rollsbattery.com/en/support/solutions/articles/4345-agm-charging>.
- [9] "Agm discharge characteristics," <https://support.rollsbattery.com/en/support/solutions/articles/4346-agm-discharge-characteristics>.
- [10] R. Aleliunas, R. M. Karp, R. J. Lipton, L. Lovasz, and C. Rackoff, "Random walks, universal traversal sequences, and the complexity of maze problems," in *Proceedings of the 20th Annual Symposium on Foundations of Computer Science*. USA: IEEE Computer Society, 1979. [Online]. Available: <https://doi.org/10.1109/SFCS.1979.34>